



Technische
Universität
Braunschweig

BACHELOR THESIS

Write down your Title right here!

Fabian Alich
(Matrikel)
(Programme)

Institute of Operating Systems and Computer Networks
Algorithms Division

First reviewer
(Anon Anonymous, Institute of XY)

Second reviewer
(Anon Anonymous, Institute of YX)

Supervised by
(Anon Anonymous)
(tba)

May 9, 2025

Statement of Originality

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, May 9, 2025

Aufgabenstellung

This is where your “Aufgabenstellung” goes. You should get this from your supervisor!

Abstract

Your abstract gives a short introduction to your thesis (context), the problem description, and a brief description of your results.

You may want to include a german abstract, too.

Contents

1	Introduction	1
1.1	Results	1
1.2	Related Work	1
1.2.1	Delaunay Triangulations	1
1.2.2	Flips in Triangulations	2
1.2.3	degree constrained minimum spanning tree	2
1.3	Overview	6
2	Preliminaries	7
3	Content	9
4	Experiments	11
5	Conclusion (and Future Work)	13

List of Figures

1.1	<i>A perspective view of a terrain</i> aus [2, p. 183]	1
1.2	<i>Examples of edge move, edge rotation, and edge slide</i> aus [1, p. 70]	3

List of Tables

1 Introduction

You can write an introduction here!

1.1 Results

We proof that the TRAVELING SALESMEN PROBLEM(TSP) is NP-complete.

... that the \textsc{Traveling Salesmen Problem}(TSP) is ...

You can use abbreviations for your problem names to avoid redundancy. Try to use existing abbreviations from related work if possible.

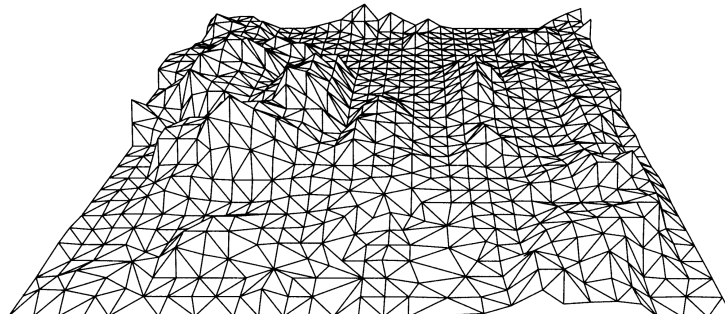
1.2 Related Work

Das ist meine Übersicht zur Related work

1.2.1 Delaunay Triangulations

Als Grundlage für den Algorithms können Delaunay Triangulation betrachtet werden. Diese Liefern eine Triangulation welche möglichst Gleichschenklige und Gleichwinklige Dreiecke nutzt [6]. Dies kann erreicht werden in dem der minimale Winkel maximiert wird. In dem Buch von de Berg et.al. [2] wird diese Variante der Triangulation genauer betrachtet. Triangulation eignen sich aufgrund ihres Aufbaues gut um Höhendaten im zwei Dimensionalen Raum darzustellen (siehe Figure 1.1). Hier kann genutzt werden das Dreiecke verschiedene Innenwinkel und Kantenlängen besitzen. Das Grundproblem ist hier das die höhendaten nur an bestimmten Punkten bekannt sind und somit die

Figure 1.1: *A perspective view of a terrain* aus [2, p. 183]



1 Introduction

Restlichen Höhendaten erraten werden müssen. Das kann durch die Delaunay Triangulation gut erreicht werden, da die Dreiecke durch ihre Gleichschenklige Eigenschaft optisch gut die fehlenden Höhendaten ergänzen können. Es wird auch genannt das die Anzahl der Kanten immer

$$3 \cdot n - 3 - k$$

und die Anzahl der Dreiecke

$$2 \cdot n - 2 - k$$

immer beträgt. Wobei n die Anzahl der Knoten und k die Anzahl der Knoten auf der Konvexen Hülle darstellt.

1.2.2 Flips in Triangulations

Flips sind Möglichkeiten, um Kanten in einer Triangulation auszutauschen. Da Triangulationen für die gleiche Knotenmenge, wie in [2] erwähnt, stets die gleiche Anzahl an Kanten besitzen, können keine Kanten hinzugefügt oder entfernt werden, um den Grad einer Triangulation zu verändern. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [3] zeigten, dass jede Triangulation mindestens

$$\frac{n - 4}{2}$$

flipbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut dem Paper *Flips in planar graphs* [1] lässt sich jede mögliche Triangulation in jede beliebige andere Triangulation überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn eine *degree-constrained triangulation* möglich ist, diese mit den nötigen Flips konstruiert werden kann.

Eine weitere Art von Flips sind sogenannte k -Flips. Diese stellen eine zusätzliche Möglichkeit dar, Kanten innerhalb einer Triangulation zu verschieben. Dabei werden k Kanten entfernt und durch k neue Kanten ersetzt. Normale Flips entsprechen somit 1-Flips. Der Vorteil von k -Flips liegt darin, dass mit einem Schritt größere strukturelle Veränderungen möglich sind. Drei beispielhafte Flips sind in Figure 1.2 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.

1.2.3 degree constrained minimum spanning tree

Eine Gradregulierung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained* ist dass ein Minimum Spanning Tree erzeugt werden soll

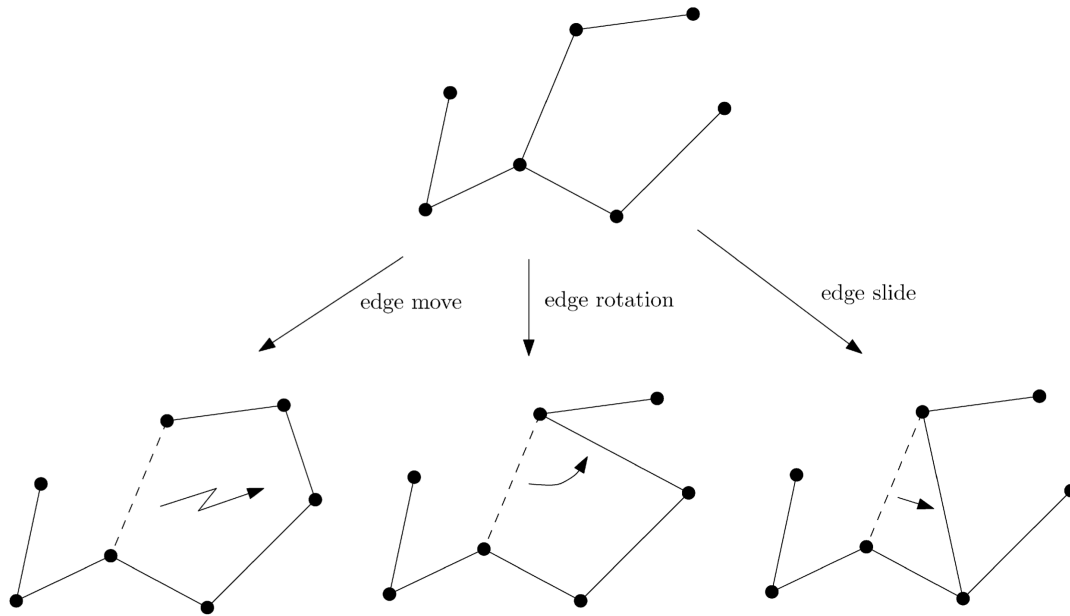


Figure 1.2: *Examples of edge move, edge rotation, and edge slide* aus [1, p. 70]

welcher an jedem Knoten einen maximalen Grad von d hat. Dabei wird d global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [4] vorgestellt welche das Problem lösen sollen. Der erste Ansatz ignoriert zu Beginn den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt welche die Gradbegrenzung nicht verletzen. Im weiteren Verlauf wird versucht die Anzahl der Kanten zu minimieren.

Der minimale spanning Tree ohne Gradbegrenzung kann in polynomialer Zeit gelöst werden. Der nächste Ansatz erzeugt zuerst einen minimalen spanning Tree welcher die Gradbegrenzung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Dieses neue Gewicht wird mit folgender Formel berechnet

$$newweight = weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

Wobei $newweight$ das neue Gewicht ist und $weight$ das aktuelle Gewicht. Die Variable $fault$ hat den Wert $\{0, 1, 2\}$ abhängig davon wie viele Knoten an der Kante gerade die Gradbegrenzung verletzen. Die Variablen min und max bezeichnen das kleinste beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler spanning Tree gebildet wobei durch das neue Gewicht nun Kanten bevorzugt werden welche keine Gradverletzung verursachen. Diese beiden Schritte können so lange wiederholt werden bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Das Problem bei diesem Ansatz ist dass Lösungen fälschlicherweise als *nicht möglich* bewertet wurden aufgrund der maximalen Iterationen.

1 Introduction

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst das Problem des degree-constrained minimum spanning tree nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf die Degree-constrained Triangulation angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt. Danach werden weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht. Bei diesen Knoten wird überprüft, ob sie gleich gute oder bessere Ergebnisse liefern. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines *greedy*-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung des lokalen Problems stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Punkt in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Punkt ausgewählt.

Dieser Ansatz wird *simulated annealing* genannt. Auch hier werden nur neue Knoten gesucht und bestehende Lösungen verbessert. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei ist $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist i und die mögliche neue Lösung ist j . Die Funktion f gibt hierbei die Kosten der Lösung an. Der Parameter c wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Die Veränderung von c bewirkt die gewünschte Änderungsrate. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$.

Der letzte Ansatz wird erst am Ende vorgestellt und beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet. Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

In dem Paper von Krishnamoorthy et.al. [5] werden Evolutionsalgorithmen noch mal genauer betrachtet. Zum einen wird die Art der Coodierung betrachtet. Es ist sinnvoll eine Art der Coodierung zu wählen mit der nur Definitions technisch richtige spanning Trees dargestellt werden können. Sonst kann es passieren das von den erstellten Lösungen viele nicht genutzt werden können und somit nicht relevant sind. Wenn eine passende Coodierung gewählt wurde, können dann nur noch der Grad und das Gewicht betrachtet werden.

Eine Möglichkeit wie Mutationen stattfinden können ist die *Crossover* Methode. Hier gibt es zwei *Eltern* Lösungen P_1 und P_2 . Von diesen werden

$$2 \cdot (n - 2)$$

verschiedene *Kinder* erzeugt. Da es $n-1$ viele Veränderungs Möglichkeiten gibt. Danach werden die *Kinder* bewertet und sortiert. Das beste Kind welches P_1 ähnelt wird ausgewählt. Sollte dieses *Kind* nun besser sein als P_1 , ersetzt es P_1 . Für P_2 wird nach dem gleichen Verfahren ein *Kind* ausgewählt.

Eine Weitere Möglichkeit ist die *Breeding* Methode. Hier gibt es einen Pool aus Lösungen. Zufällig werden Paare gebildet, welche jeweils zwei *Kinder* haben. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, das alle möglichen Veränderungen auch verändert werden.

Beide Methode sollen im Durchschnitt alle 10 Generation die beste Lösung mit allen anderen Lösungen verglichen haben.

Als mögliche Iterationsanzahl wird die Formel

$$\frac{50 \cdot n}{\bar{b}}$$

genannt. Wobei $\bar{b}$ die Durchschnittliche Grad Bedingung ist. Mit dieser Formel bekommen große Instanzen mehr Iterationen, aber leichtere Instanzen (höherer Grad) werden schneller Bearbeitet.

1.3 Overview

If needed, write down where to find everything.

2 Preliminaries

If needed, you can write down definitions here, that you need in your thesis. For this, the definition-environment can be helpful

Definition 2.1 (Graph coloring). A graph $G = (V, E)$ is k -colorable if there is a function $C : V \rightarrow \{1, \dots, k\}$ such that $C(u) \neq C(v)$ for any edge $\{u, v\} \in E$

The *chromatic number* of a graph G is the smallest number k such that G is k -colorable.

3 Content

From here, start your main content. You can always use new chapters if needed. Use proper environments for your content, e.g. theorem, definition, lemma, etc. Here are some examples.

Theorem 3.1. *Every planar graph is 4-colorable.*

Lemma 3.2. *Every bipartite graph is 2-colorable.*

Corollary 3.3. *Every tree graph is 2-colorable.*

Because graphs with at least one edge need at least two colors, we obtain the following theorem.

Theorem 3.4. *The chromatic number of every (non-trivial) bipartite graph is 2.*

4 Experiments

If you have experiments, write down:

- Implementation details (e.g., libraries, third-party programs, ...)
- In which environment are you testing (e.g., system specifications)
- What are you testing? (e.g., runtime, space needed, fps, ...)
- What are the results? Show fancy figures!
- What does the results mean? (e.g., give a little discussion on your results)
- ...

5 Conclusion (and Future Work)

This is the end of your thesis! Give a summary of what you did.

You can also write down possible future work you or other people could look at.

Bibliography

- [1] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [2] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [3] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, SCG '96, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [4] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [5] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [6] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.